

Notes:

- HW3 graded, solutions posted
- Opened issue on github repo if score on that part was not 30/30
- HW4 due today
- HW5 posted, due Nov 18
- No class next Tuesday; held on Wednesday instead!

Last class:

- Different types of plots: histograms, line graphs, heat maps
- Python functions and arguments

When we start working with functions, have to be careful of variables defined inside or outside of our functions. Consider:

```
In [1]: shift = 32
def f_to_c(temp_f):
    temp_c = (temp_f - shift)*(5/9)
    return temp_c
```

```
In [2]: print(f"68 degrees F is {f_to_c(68)} degrees C")

68 degrees F is 20.0 degrees C
```

```
In [3]: print(temp_c)
```

```

NameError                                Traceback (most recent call last)
<cell In[3], line 1
----> 1 print(temp_c)

NameError: name 'temp_c' is not defined

The variable 'temp_c' exists only in the scope of the function 'f_to_c'. We can print it there, but not outside
```

```
In [4]: shift = 32
def f_to_c_print(temp_f):
    temp_c = (temp_f - shift)*(5/9)
    print(temp_c)
    return temp_c
```

```
In [5]: f_to_c_print(68)

20.0
```

```
Out[5]: 20.0
```

```
In [6]: print(temp_c)
```

```

NameError                                Traceback (most recent call last)
<cell In[6], line 1
----> 1 print(temp_c)

NameError: name 'temp_c' is not defined

In contrast, 'shift' is a global variable that exists outside the scope of these functions
```

```
In [7]: print(shift)

32

Note that changing 'shift' changes the behaviour of the functions:
```

```
In [8]: shift = 50
print(f_to_c(68))

10.0
```

Whenever we run a program that depends on a global variable, if we change that variable, we change the behaviour of the program

Note, however, that we can set variables within our functions

```
In [9]: def f_to_c_shift(temp_f):
    shift = 32
    temp_c = (temp_f - shift)*(5/9)
    return temp_c
```

```
In [10]: f_to_c_shift(68)

20.0
```

```
Out[10]: 20.0
```

```
In [11]: print(shift)

50
```

When we define 'shift' inside the function, python uses the local value rather than the global one inside the function, but this does not change the global value.

To also change the value of 'shift' outside the function, we tell Python that we identify the local variable with the global one using the 'global' keyword:

```
In [12]: def f_to_c_global_shift(temp_f):
    global shift
    shift = 32
    temp_c = (temp_f - shift)*(5/9)
    return temp_c
```

```
In [13]: f_to_c_global_shift(68)

20.0
```

```
Out[13]: 20.0
```

```
In [14]: print(shift)

32
```

```
In [15]: shift = 50
print(shift)
print(f_to_c(68))
print(f_to_c_global_shift(68))
print(shift)
```

```
50
10.0
20.0
32
```

This is generally not a good practice. If we modify a variable inside a function and want to use it globally, generally better to explicitly declare it as input and output parameters for the function

```
In [16]: def f_to_c_shift_argument(temp_f, shift):
    shift = 32
    temp_c = (temp_f - shift)*(5/9)
    return temp_c, shift
```

```
In [19]: shift = 50
print(shift)
temp_c, shift = f_to_c_shift_argument(68, shift)
print(temp_c)
print(shift)
```

```
50
20.0
32
```

Don't have to reassign output value to shift:

```
In [20]: shift = 50
print(shift)
temp_c, shift2 = f_to_c_shift_argument(68, shift)
print(temp_c)
print(shift)
print(shift2)
```

```
50
20.0
50
32
```

```
In [21]: shift = 50
print(shift)
temp_c, _ = f_to_c_shift_argument(68, shift)
print(temp_c)
print(shift)
```

```
50
20.0
50
```

A better way of doing this is to specify default parameters that you can override:

```
In [22]: def f_to_c_default(temp_f, shift = 32):
    temp_c = (temp_f - shift)*(5/9)
    return temp_c
```

```
In [23]: print(f_to_c_default(68))
print(f_to_c_default(68, 32))
print(f_to_c_default(68, 50))
print(f_to_c_default(68))

20.0
20.0
10.0
20.0
```

Use default values to help "declutter" our function calls, by only setting needed and non-default values

Can specify variable numbers of arguments in 2 ways:

1. use `*args` to represent an arbitrary number of parameters that aren't named, and are translated from a list of arguments to a tuple
2. use `**kwargs` to represent an arbitrary number of named (keyword) arguments, that are translated to a dictionary

```
In [24]: def f(a = 1, *args, **kwargs):
    print(a)
    print(f"args is {args}")
    for num in args:
        print(a+num)
    print(f"kwargs is {kwargs}")
    for key, value in kwargs.items():
        print(f"kwargs with key {key} has value {value}")
```

```
In [27]: f(2, 1, 2, 3, 4, 5, frogs = "bogs", cats = "hats", seven = 11)
```

```
2
args is (1, 2, 3, 4, 5)
2
4
8
16
32
kwargs is {'frogs': 'bogs', 'cats': 'hats', 'seven': 11}
kwarg with key frogs has value bogs
kwarg with key cats has value hats
kwarg with key seven has value 11

Not a lot of situations where *args is useful:

• it is easier to read than a long list of parameters, but much less intuitive. Useful (and justified) when you are going to do the same thing with all these arguments, but generally better to have one argument given as a tuple, than an arbitrary tuple like this
```

In contrast, many situations where using `**kwargs` is useful. Very helpful when one function takes parameters that are passed on to another function. (E.g., as we often do for plotting with matplotlib or other packages)

Also find uses for `**kwargs` in object-oriented programming

By now, we have a number of data types that we know how to use: `int`, `float`, `string`, `list`, `dict`.

Can also create new data types, using classes

Think about a task that we may want to do, and aim to build a custom data type to support it.

As an example, lets build a data type for working with polynomials, $p(x) = \sum_{i=0}^d a_ix^{i^{\text{th}}}$ with real-valued coefficients $\{a_i\}_{i=0}^d$

To store such a polynomial, we need to store the coefficients. To do this, we create an initialization method for a class:

```
In [28]: class Polynomial:
    def __init__(self, coeffs):
        self.coefficients = coeffs
```

Use `self` to store data in a class, and `__init__` to initialize a class

```
In [29]: p = Polynomial((1,2,3))
p.coefficients
```

```
Out[29]: (1, 2, 3)
```

Note: nothing in the class that tells us how to interpret the data. Write some documentation to tell us what it means, and some methods that act on the data

```
In [30]: class Polynomial:
    """
    Perform operations with polynomials

    Stores a polynomial in terms of its coefficients, ordered from lowest order first,
    so that self.coefficients[i] is coefficient of x^i.
    """
    def __init__(self, coeffs):
        self.coefficients = coeffs

    def degree(self):
        return len(self.coefficients)-1
```

Just as for functions and loops, context in classes is determined by indentation. Functions defined inside a class are called methods. Some methods are for internal use by python and denoted by double underscores before and after method name, like `__init__`. Sometimes call these "dunder" methods, noting they are generally never called directly by a user.

The data that we store in a class are called the "attributes" of the class. Set these by storing them as `self.attribute_name = data`

Now think about what methods might be useful:

```
In [31]: class Polynomial:
    """
    Perform operations with polynomials

    Stores a polynomial in terms of its coefficients, ordered from lowest order first,
    so that self.coefficients[i] is coefficient of x^i.
    """
    def __init__(self, coeffs):
        self.coefficients = coeffs

    def degree(self):
        return len(self.coefficients)-1
```

Methods always take `self` as their first argument, meaning the data associated with the object itself, then can have other arguments. Don't have to call this self, but no need to mess with this convention

```
In [32]: p = Polynomial((1,2,3,4))
p.degree()
```

```
Out[32]: 3
```

Note: Cannot have both a method and an attribute "share" a name. Each piece of a class must have a unique name

Can we print a polynomial in a reasonable way?

```
In [33]: print(p)

<_main_.Polynomial object at 0x11fce4ad0>

Just class name and a memory address, so not useful.
```

How do we tell Python how to print a polynomial object. Define a method. By default, Python looks for a method named `__str__` when print is called on an object. Could define this simply:

```
In [34]: class Polynomial:
    """
    Perform operations with polynomials

    Stores a polynomial in terms of its coefficients, ordered from lowest order first,
    so that self.coefficients[i] is coefficient of x^i.
    """
    def __init__(self, coeffs):
        self.coefficients = coeffs

    def degree(self):
        return len(self.coefficients)-1

    def __str__(self):
        return f"Polynomial object with coefficients {self.coefficients}"
```

```
In [35]: p = Polynomial((2, 3, 7, 6))
print(p)

Polynomial object with coefficients (2, 3, 7, 6)
```

This `print` function only impacts Polynomial objects:

```
In [36]: print("hello world")

hello world

Can also work harder. Recognize that one special case is a zero polynomial, so write a method to check this, and then use it in our __str__ method:
```

```
In [39]: class Polynomial:
    """
    Perform operations with polynomials

    Stores a polynomial in terms of its coefficients, ordered from lowest order first,
    so that self.coefficients[i] is coefficient of x^i.
    """
    def __init__(self, coeffs):
        self.coefficients = coeffs

    def degree(self):
        return len(self.coefficients)-1

    def is_zero(self):
        zero = True
        for val in self.coefficients:
            if val != 0:
                zero = False
                break
        return zero

    def __str__(self):
        terms = []
        deg = self.degree()
        coeffs = self.coefficients

        if not self.is_zero():
            if deg > 1:
                for i in range(deg,1,-1):
                    if coeffs[i] != 0:
                        terms += [f"{coeffs[i]}x^{i}"]

            if deg > 0:
                if coeffs[1] != 0:
                    terms += [f"{coeffs[1]}x"]
                if coeffs[0] != 0:
                    terms += [f"{coeffs[0]}"]
            else:
                terms += ["0"]

        return f"Polynomial p(x) = " + " + ".join(terms)

    def __repr__(self):
        return type(self).__name__ + "(" + repr(self.coefficients) + ")"

    __repr__ method should give the string needed to recreate the object in one command. Note type(self).__name__ just gives back the name of the class
```

```
In [40]: p = Polynomial((2, 4, 6, 8))
print(p)

Polynomial p(x) = 8x^3 + 6x^2 + 4x + 2

A second method for printing is the __repr__ method, which provides the output when we just type a variable name at the command line:
```

```
In [41]: class Polynomial:
    """
    Perform operations with polynomials

    Stores a polynomial in terms of its coefficients, ordered from lowest order first,
    so that self.coefficients[i] is coefficient of x^i.
    """
    def __init__(self, coeffs):
        self.coefficients = coeffs

    def degree(self):
        return len(self.coefficients)-1

    def is_zero(self):
        zero = True
        for val in self.coefficients:
            if val != 0:
                zero = False
                break
        return zero

    def __str__(self):
        terms = []
        deg = self.degree()
        coeffs = self.coefficients

        if not self.is_zero():
            if deg > 1:
                for i in range(deg,1,-1):
                    if coeffs[i] != 0:
                        terms += [f"{coeffs[i]}x^{i}"]

            if deg > 0:
                if coeffs[1] != 0:
                    terms += [f"{coeffs[1]}x"]
                if coeffs[0] != 0:
                    terms += [f"{coeffs[0]}"]
            else:
                terms += ["0"]

        return f"Polynomial p(x) = " + " + ".join(terms)

    def __repr__(self):
        return type(self).__name__ + "(" + repr(self.coefficients) + ")"

    __repr__ method should give the string needed to recreate the object in one command. Note type(self).__name__ just gives back the name of the class
```

```
In [42]: p = Polynomial((2,4,6,8))
p

Out[42]: Polynomial((2, 4, 6, 8))
```

```
In [43]: z = Polynomial((0,0,0))
print(z)

Polynomial p(x) = 0

How do we tell if 2 polynomials are the same? Normally use ==
```

```
In [44]: val1 = 2
val2 = 2
print(val1 == val2)
print(val1 == val2 + 2)

True
False

Try this with Polynomial class:
```

```
In [45]: p1 = Polynomial((1,2,3))
p2 = Polynomial((1,2,3))
print(p1 == p2)

False

Why doesn't this work? Because we have 2 different objects, and python doesn't know how to compare them.

We define this with the __eq__ method:
```

```
In [46]: class Polynomial:
    """
    Perform operations with polynomials

    Stores a polynomial in terms of its coefficients, ordered from lowest order first,
    so that self.coefficients[i] is coefficient of x^i.
    """
    def __init__(self, coeffs):
        self.coefficients = coeffs

    def degree(self):
        return len(self.coefficients)-1

    def is_zero(self):
        zero = True
        for val in self.coefficients:
            if val != 0:
                zero = False
                break
        return zero

    def __str__(self):
        terms = []
        deg = self.degree()
        coeffs = self.coefficients

        if not self.is_zero():
            if deg > 1:
                for i in range(deg,1,-1):
                    if coeffs[i] != 0:
                        terms += [f"{coeffs[i]}x^{i}"]

            if deg > 0:
                if coeffs[1] != 0:
                    terms += [f"{coeffs[1]}x"]
                if coeffs[0] != 0:
                    terms += [f"{coeffs[0]}"]
            else:
                terms += ["0"]

        return f"Polynomial p(x) = " + " + ".join(terms)

    def __repr__(self):
        return type(self).__name__ + "(" + repr(self.coefficients) + ")"

    def __eq__(self, other):
        return isinstance(other, Polynomial) and self.coefficients == other.coefficients
```

Now can compare:

```
In [47]: p1 = Polynomial((1,2,3))
p2 = Polynomial((1,2,3))
print(p1 == p2)

True
```

```
In [48]: print(p1 == "Polynomial")

False
```

Recognize that this a form of *operator overloading*. Whenever we type `a == b`, we actually evaluate `a.__eq__(b)` if that method exists. Same for other arithmetic operations.

Thus, we define `__add__` to determine the sum of two polynomials:

Idea: Given 2 tuples of coefficients (possibly of different lengths), add the common coefficients, then append those of longer one to get coefficients of their sum

```
In [49]: class Polynomial:
    """
    Perform operations with polynomials

    Stores a polynomial in terms of its coefficients, ordered from lowest order first,
    so that self.coefficients[i] is coefficient of x^i.
    """
    def __init__(self, coeffs):
        self.coefficients = coeffs

    def degree(self):
        return len(self.coefficients)-1

    def is_zero(self):
        zero = True
        for val in self.coefficients:
            if val != 0:
                zero = False
                break
        return zero

    def __str__(self):
        terms = []
        deg = self.degree()
        coeffs = self.coefficients

        if not self.is_zero():
            if deg > 1:
                for i in range(deg,1,-1):
                    if coeffs[i] != 0:
                        terms += [f"{coeffs[i]}x^{i}"]

            if deg > 0:
                if coeffs[1] != 0:
                    terms += [f"{coeffs[1]}x"]
                if coeffs[0] != 0:
                    terms += [f"{coeffs[0]}"]
            else:
                terms += ["0"]

        return f"Polynomial p(x) = " + " + ".join(terms)

    def __repr__(self):
        return type(self).__name__ + "(" + repr(self.coefficients) + ")"

    def __eq__(self, other):
        return isinstance(other, Polynomial) and self.coefficients == other.coefficients

    def __add__(self, other):
        if isinstance(other, Polynomial):
            low_degree = min(self.degree(), other.degree())
            scoefs = self.coefficients
            ocoefs = other.coefficients

            coeffs = tuple(sc + oc for sc, oc in zip(scoefs[low_degree+1:], ocoefs[low_degree+1]))
            coeffs += scoefs[low_degree+1:] + ocoefs[low_degree+1:]

            return Polynomial(coeffs)
        else:
            return NotImplementedError
```

```
In [50]: p1 = Polynomial((1,2,3))
p2 = Polynomial((1,2,3,4))
print(p1 + p2)

Polynomial p(x) = 4x^3 + 6x^2 + 4x + 2

What if second Polynomial is just a number?

One way:



```
In [51]: p1 = Polynomial((1,2,3))
p2 = Polynomial((7,))
print(p1+p2)

Polynomial p(x) = 3x^2 + 2x + 8

Want to be able to write p1+p2
```


```